

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
import matplotlib.pyplot as plt
import seaborn as sns
```

Mount Colab to Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Input File

```
Housing = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/Housi
```

```
Housing.head()
```

	ID	Decision	Build_Selfcon	Tier	Accommodation_Class	Marital
0	AJM-39R8-519413	0	Self Contruction	3	Non_Rented	
1	AJM-3A3Z-876970	0	Self Contruction	3	Non_Rented	
2	AJM-7427-952085	1	Self Contruction	3	Non_Rented	
3	AJM-8ZTZ-979040	1	Self Contruction	3	Rented	
4	AJM-9SP2-614810	1	Self Contruction	3	Non_Rented	

5 rows x 22 columns

```
Housing.isna().sum()
```

	0
ID	0
Decision	0
Build_Selfcon	0
Tier	0
Accommodation_Class	0
Marital_Status	0
Employer_Type	0
OldEmi_d	0
BankSave_d	0
Age	0
YrsAdd	0
YrsJob	0
TotInc	0
MarVal	0
LoanReq	0
Term	0
CalcEmi	0
IIR	0
IAR	0
OldEmi	0
LTV	0
FOIR	0

dtype: int64

```
Housing.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1570 entries, 0 to 1569
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ID                                     1570 non-null   object
1   Decision                             1570 non-null   int64
2   Build_Selfcon                         1570 non-null   object
3   Tier                                 1570 non-null   int64
4   Accommodation_Class                  1570 non-null   object
5   Marital_Status                       1570 non-null   object
6   Employer_Type                       1570 non-null   object
7   OldEmi_d                             1570 non-null   int64
8   BankSave_d                           1570 non-null   int64
9   Age                                  1570 non-null   int64
10  YrsAdd                               1570 non-null   int64
11  YrsJob                               1570 non-null   int64
12  TotInc                               1570 non-null   int64
13  MarVal                               1570 non-null   int64
14  LoanReq                              1570 non-null   int64
15  Term                                 1570 non-null   int64
16  CalcEmi                             1570 non-null   float64
17  IIR                                  1570 non-null   float64
18  IAR                                  1570 non-null   float64
19  OldEmi                              1570 non-null   int64
20  LTV                                  1570 non-null   float64
21  FOIR                                 1570 non-null   float64
dtypes: float64(5), int64(12), object(5)
memory usage: 270.0+ KB
```

```
Housing['Marital_Status'].value_counts()
```

	count
Marital_Status	
Married	1416
Unmarried	154

dtype: int64

▼ Convert Binary Varibale

```
Housing['Marital_Status'].replace({'Married':1,'Unmarried':0},inpla
```

```
/tmp/ipython-input-393966625.py:1: FutureWarning: A value is trying  
The behavior will change in pandas 3.0. This inplace method will nev
```

```
For example, when doing 'df[col].method(value, inplace=True)', try u
```

```
Housing['Marital_Status'].replace({'Married':1,'Unmarried':0},inpl  
/tmp/ipython-input-393966625.py:1: FutureWarning: Downcasting behavi  
Housing['Marital_Status'].replace({'Married':1,'Unmarried':0},inpl
```

✓ Dummy Variable

```
Dummy = pd.get_dummies(Housing['Employer_Type'],prefix="Emp_Type").  
Dummy1= pd.get_dummies(Housing['Tier'],prefix="Tier").astype(int)  
Housing=pd.concat( [Housing,Dummy,Dummy1],axis=1)
```

```
Housing.head()
```

	ID	Decision	Build_Selfcon	Tier	Accommodation_Class	Marital
0	AJM-39R8-519413	0	Self Contruction	3	Non_Rented	
1	AJM-3A3Z-876970	0	Self Contruction	3	Non_Rented	
2	AJM-7427-952085	1	Self Contruction	3	Non_Rented	
3	AJM-8ZTZ-979040	1	Self Contruction	3	Rented	
4	AJM-9SP2-614810	1	Self Contruction	3	Non_Rented	

5 rows x 29 columns

```
list[Housing.columns]
```

```
list[Index(['ID', 'Decision', 'Build_Selfcon', 'Tier',  
'Accommodation_Class',  
          'Marital_Status', 'Employer_Type', 'OldEmi_d', 'BankSave_d',  
'Age',  
          'YrsAdd', 'YrsJob', 'TotInc', 'MarVal', 'LoanReq', 'Term',  
'CalcEmi',  
          'IIR', 'IAR', 'OldEmi', 'LTV', 'FOIR', 'Emp_Type_Business',  
          'Emp_Type_Corporate', 'Emp_Type_Govt',  
'Emp_Type_Ind/SmallBus',  
          'Tier_1', 'Tier_2', 'Tier_3'],  
         dtype='object'])]
```

```
Col=['Marital_Status', 'OldEmi_d', 'BankSave_d', 'Age', 'YrsAdd', '  
      'TotInc', 'MarVal', 'LoanReq', 'Term', 'CalcEmi', 'IIR', 'IA  
      'OldEmi', 'LTV', 'FOIR', 'Emp_Type_Business', 'Emp_Type_Corp  
      'Emp_Type_Govt', 'Emp_Type_Ind/SmallBus', 'Tier_1', 'Tier_2']
```

```
x=Housing[Col]  
y=Housing.Decision
```

✓ Train Test Split

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.3,
```

✓ Building Decision Tree

```
clf = DecisionTreeClassifier (criterion='entropy',max_depth=3)
```

```
model = clf.fit(x_train,y_train)
```

```
y_pred = model.predict(x_test)
```

```
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.8152866242038217
```

✓ Draw Tree

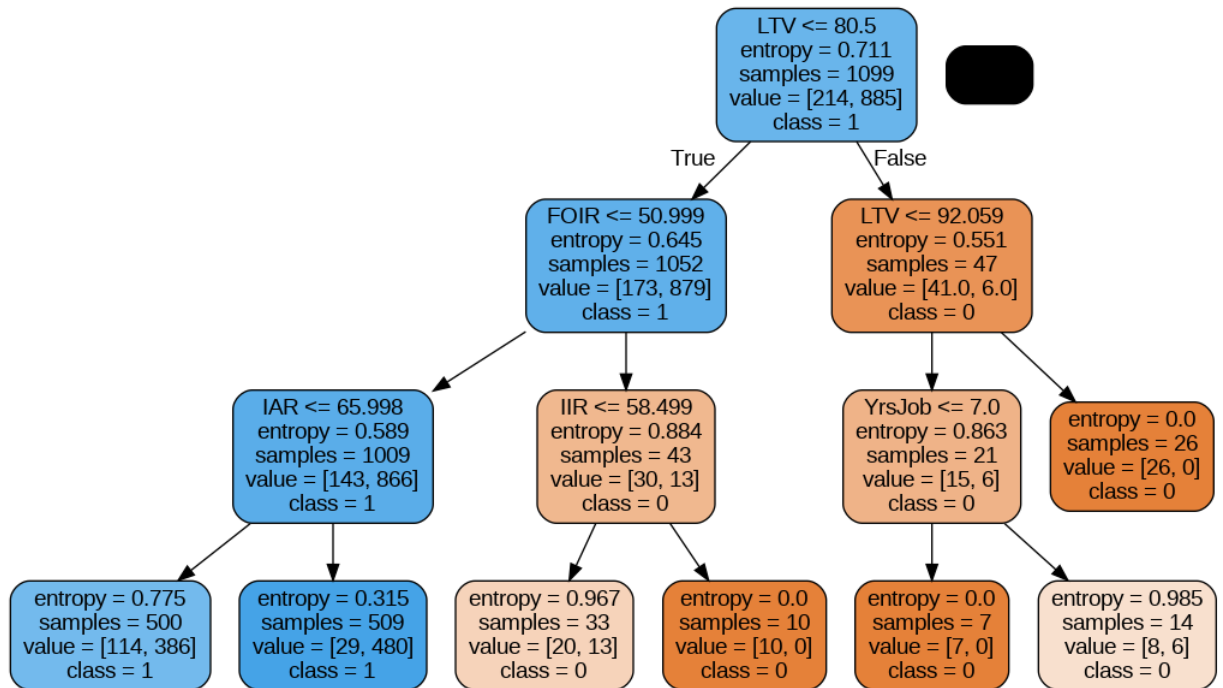
```

### from textwrap import fill
from pydotplus import graph_from_dot_data
from sklearn.tree import export_graphviz
### from sklearn import tree
from IPython.display import Image

dot_data = export_graphviz(clf,out_file=None,feature_names=Col,fill

graph = graph_from_dot_data(dot_data)
graph.write_png('tree.png')
Image(graph.create_png())

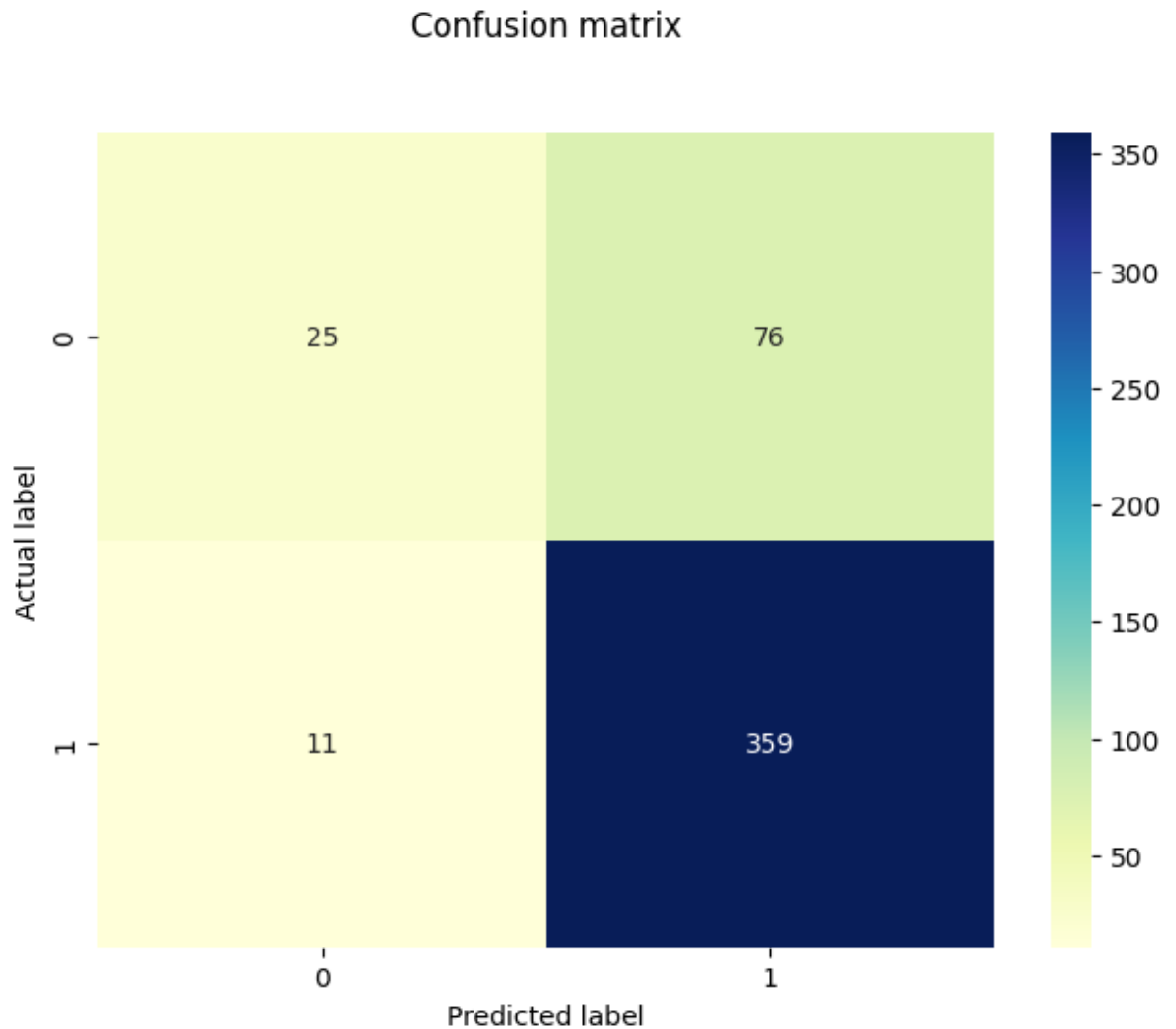
```



✓ Confusion Matrix

```
cnf_matix= metrics.confusion_matrix(y_test,y_pred)
sns.heatmap(pd.DataFrame(cnf_matix), annot=True, cmap="YlGnBu" ,fmt
plt.tight_layout()
plt.title('Confusion matrix', y=1.1)
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
```

```
Text(0.5, 23.52222222222222, 'Predicted label')
```



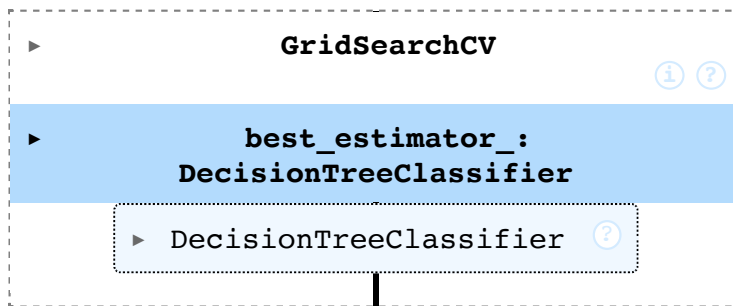
Hyperparameter Tunning ##Step 1

```
parameters={
    "max_depth": [1,2,3,4,5,6,7,8,9,10,11,12],
    "criterion": ["gini","entropy"]
}
```

```
clf1=DecisionTreeClassifier()
```

```
from sklearn.model_selection import GridSearchCV
grid=GridSearchCV(clf1,param_grid=parameters,cv=10,scoring='accuracy
```

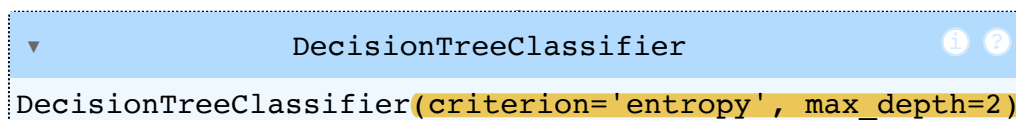
```
grid.fit(x_train,y_train)
```



```
grid.best_score_
```

```
np.float64(0.8453628023352794)
```

```
grid.best_estimator_
```



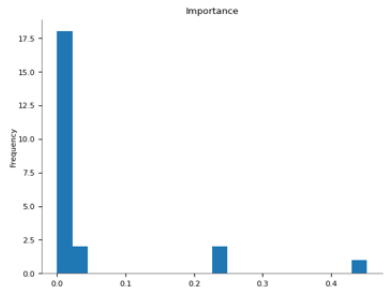
```
feat_dict={}
for col, val in sorted(zip(x_train.columns,model.feature_importance
                        key=lambda x:x[1],reverse=True)):
    feat_dict[col]=val
```

```
feat_df = pd.DataFrame.from_dict(feat_dict,orient='index',columns=[
feat_df
```

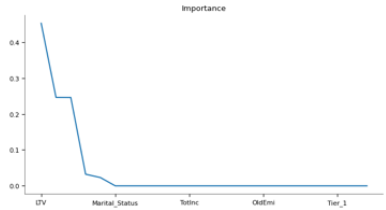
	Importance
LTV	0.452484
FOIR	0.246157
IAR	0.245945
IIR	0.032393
YrsJob	0.023020
Marital_Status	0.000000
OldEmi_d	0.000000
BankSave_d	0.000000
Age	0.000000

YrsAdd	0.000000
TotInc	0.000000
MarVal	0.000000
LoanReq	0.000000
Term	0.000000
CalcEmi	0.000000
OldEmi	0.000000
Emp_Type_Business	0.000000
Emp_Type_Corporate	0.000000
Emp_Type_Govt	0.000000
Emp_Type_Ind/SmallBus	0.000000
Tier_1	0.000000
Tier_2	0.000000
Tier_3	0.000000

Distributions



Values



```
Housing['Decision'].value_counts()
```

	count
Decision	
1	1255
0	315

dtype: int64

✓ Class Imbalance Problem (SMOTE Method)

```
pip install imbalanced-learn
```

```
Requirement already satisfied: imbalanced-learn in /usr/local/lib/py  
Requirement already satisfied: numpy<3,>=1.25.2 in /usr/local/lib/py  
Requirement already satisfied: scipy<2,>=1.11.4 in /usr/local/lib/py  
Requirement already satisfied: scikit-learn<2,>=1.4.2 in /usr/local/  
Requirement already satisfied: joblib<2,>=1.2.0 in /usr/local/lib/py  
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /usr/local
```

```
from imblearn.over_sampling import SMOTE  
sm= SMOTE(random_state=11)  
x_res,y_res=sm.fit_resample(x_train,y_train)
```

```
print(pd.Series(y_res).value_counts())  
print(pd.Series(y_train).value_counts())
```

```
Decision  
1      885  
0      885  
Name: count, dtype: int64  
Decision  
1      885  
0      214  
Name: count, dtype: int64
```

```
clf2 = DecisionTreeClassifier (criterion='entropy',max_depth=2)  
model2 = clf2.fit(x_res,y_res)  
y_pred2 = model2.predict(x_test)
```

```
from sklearn.metrics import classification_report, confusion_matrix
cm= confusion_matrix(y_test,y_pred2)
cm_df = pd.DataFrame(cm,columns=['Predicted 0','Predicted 1'],index
cm_df
```

	Predicted 0	Predicted 1
Actual 0	78	23
Actual 1	229	141

```
print(classification_report(y_test,y_pred2,digits=2))
```

	precision	recall	f1-score	support
0	0.25	0.77	0.38	101
1	0.86	0.38	0.53	370
accuracy			0.46	471
macro avg	0.56	0.58	0.46	471
weighted avg	0.73	0.46	0.50	471